DiffuserCam: Physics-Informed Reconstruction Transfer Across Cameras

Data:

- DiffuserCam Lensless Mirflickr Dataset
- bezzam/DiffuserCam-Lensless-Mirflickr-Dataset-NORM

Waller Lab DiffuserCam: DiffuserCam

Papers:

- [1710.02134] DiffuserCam: Lensless Single-exposure 3D Imaging
- [1908.11502] Learned reconstructions for practical mask-based lensless imaging
- [2403.00537] A Modular and Robust Physics-Based Approach for Lensless Image Reconstruction
- [2206.01430] LenslessPiCam: A Hardware and Software Platform for Lensless Computational Imaging with a Raspberry Pi
- [1912.10557] Algorithm Unrolling: Interpretable, Efficient Deep Learning for Signal and Image Processing

Goal: Build a lensless camera (Raspberry Pi sensor with its lens replaced by Scotch tape) and test whether physics-informed reconstruction networks trained on one lensless camera generalize to a new one.

Background: A lensless camera's measurement is the scene convolved with the diffuser's point spread function (PSF). Classical reconstruction solves a regularized inverse problem with ADMM, which is slow and requires hand-tuned parameters. An unrolled network is an iterative algorithm turned into a neural network. Each iteration becomes a layer, allowing hand-tuned settings to become weights learned from the data.'

Question: Does an unrolled network trained on one camera transfer to a new camera by swapping in a new PSF?

Hypothesis: Le-ADMM will transfer with modest degradation since it reads the new PSF explicitly. Le-ADMM-U will transfer worse since its U-Net learns the old camera's artifacts. The pure U-Net will fail to generalize since it memorizes the training PSF.

Build:

1) ADMM solver with TV regularization and nonnegativity
2) Le-ADMM and Le-ADMM-U (unrolled, 5 iterations), trained on the DiffuserCam dataset
3) A pure U-Net baseline with no physics
4) Stretch goal: My own tape-based camera with calibrated PSF

Experiments:

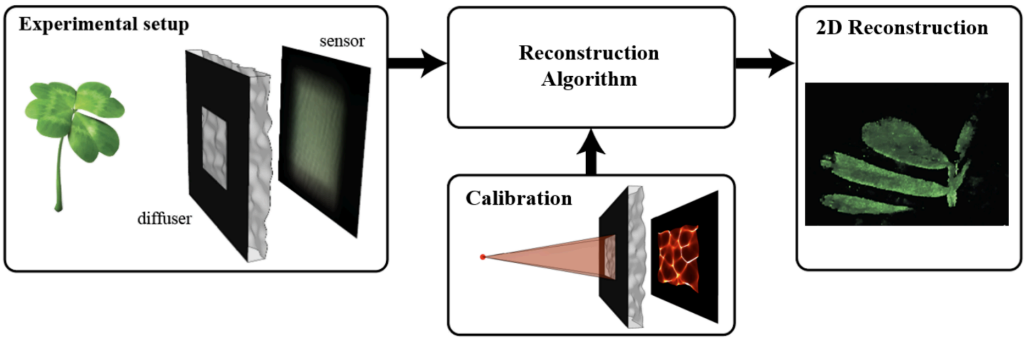
1) Reproduce via train and test on the DiffuserCam dataset
2) Simulated transfer w/ dataset images convolved with a new PSF + noise profile

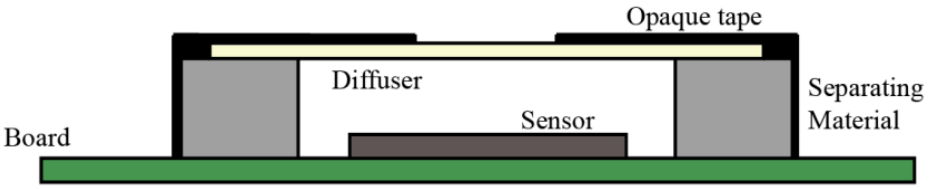
3) Real transfer w/ own camera and imaging pictures displayed on monitor

Evaluation: Reconstruction quality (PSNR, SSIM, LPIPS) and speed.

https://waller-lab.github.io/DiffuserCam/tutorial/building_guide.pdf

What is a DiffuserCam: a diffuser with an aperture that is placed in front of a camera sensor.

- The diffuser scatters light but <u>does NOT</u> absorb it.
    - The algorithm requires the point spread function (PSF) → the reconstruction algorithm uses the PSF to recover the original scene from raw sensor data
    - 

    - 

Requirements:

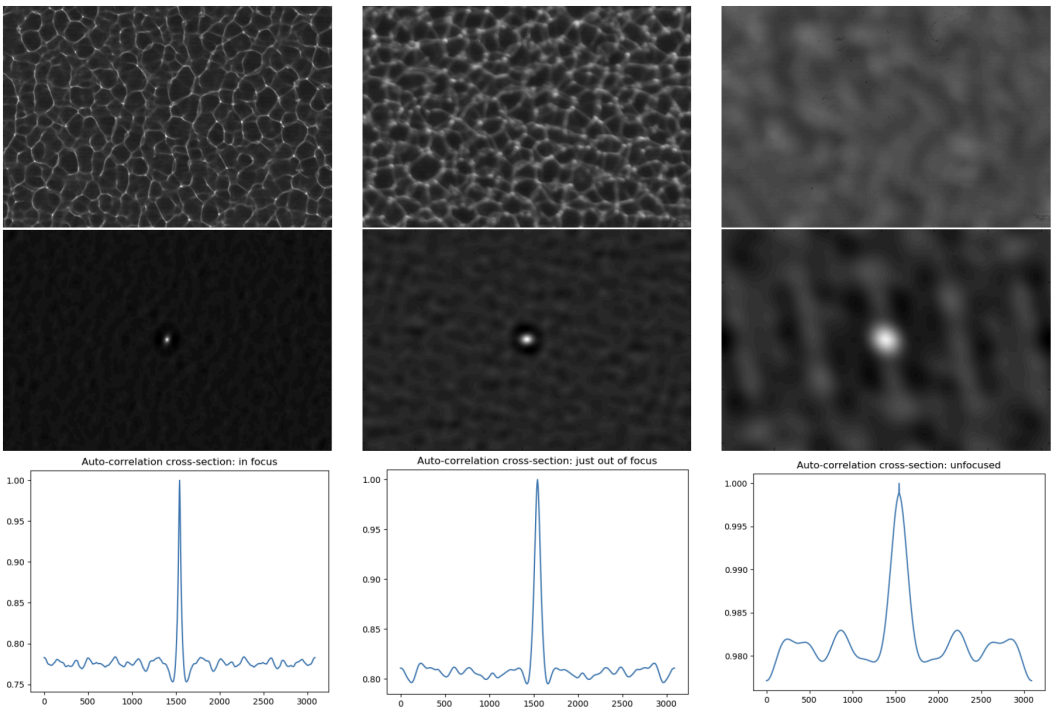
- Digital sensor:
- Diffuser: double-sided scotch tape
- Opaque separating material: black foamboard, cardboard, paper
- Black tape:
- Point source: LED

To build the DiffuserCam:

- Find the “focal distance” of the diffuser
- Place separating material around the sensor and tape the diffuser onto it
- Use black tape to create an aperture
- Take a calibration image (PSF) of a point source
- Take an image of your object, well illuminated
- Run the reconstruction algorithm with the PSF and object images as inputs

<u>Find the optimal focal distance</u>

The focal distance of the diffuser is the distance at which the PSF has the sharpest features. It was found that a 3mm distance + double-sided tape diffuser creates the sharpest caustic.



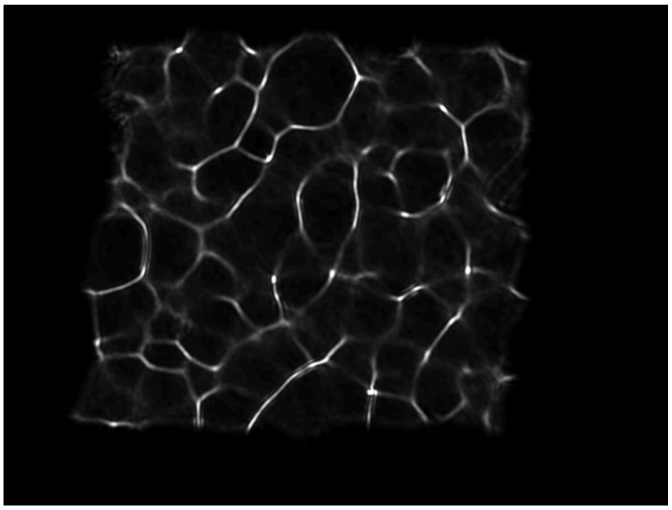
(a) Focused (b) Almost focused (c) Out of focus

Figure 4: Sensor reading (top) and autocorrelation (middle) of a point light source when (a) the diffuser is placed at the right distance, (b) the diffuser is too far from the sensor, (c) the diffuser is very far from the sensor. Notice that the width central peak of the autocorrelations increase as the diffuser is moved out of focus. This is most evident in the cross-sections (bottom) of the autocorrelations in figures (a), (b), (c). In addition, the surrounding side-lobes in the autocorrelation become more pronounced as the diffuser becomes more out of focus.

Place an aperture (hole) on the diffuser

Without the aperture, we wouldn't be able to calibrate using a single image because movement of the point source would allow new caustic to come into view on the sensor.

[!!!] Make sure that the entire cropped PSF is within the sensor field of view (i.e., black frame around caustic)



Calibrate the sensor

For single-shot calibration, we want the point source with the right brightness and smallest size possible. It should use the full dynamic range of the sensor but not saturate it. In particular, no pixels should have a value equal to their maximum value because that loses the true value beyond 255.

To optimize the caustic, we can attach the LED to a tunable voltage source to adjust brightness levels. We can also vary exposure time of the sensor. Additionally, we could also use a histogram of the values at each pixel.

Use a pinhole aperture made of cardboard + LED.

Take the images

The object should be illuminated well. Stray reflections from other objects can show up as noise or blobs so a dark background is ideal.

Try taking pictures of an image on the iPhone first.

Reconstructing images

First, navigate to the directory in which you want to place all the diffuser cam code. Then, enter the following commands:

```
git clone https://github.com/Waller-Lab/DiffuserCam-Tutorial.git
cd DiffuserCam-Tutorial
conda env create
source activate diffuser_cam
```

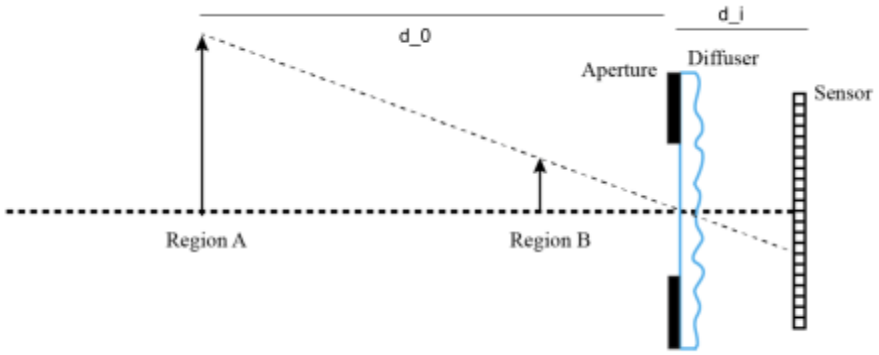
If using windows, use `activate diffuser_cam` instead.

Next, open up the `admm_config.yml` or `gd_config.yml` files and modify the parameters as necessary. Please see the corresponding jupyter notebooks for explanations of those parameters.

Finally, run `python admm.py` or `python gd.py` to reconstruct the images you selected in the config file.

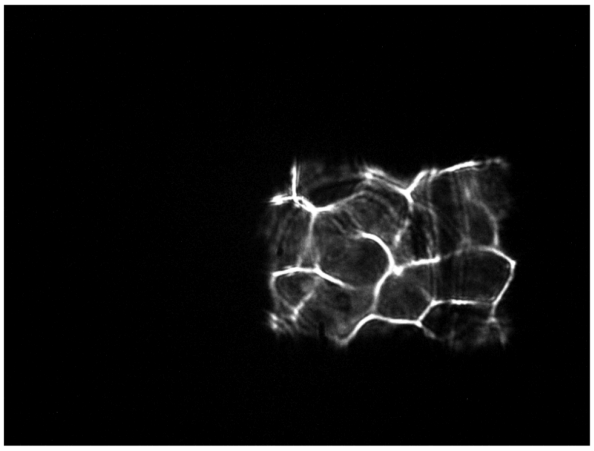
Analyzing the diffuser

1) Focal distance
    a) Magnification is given by: $m = \frac{-d_i}{d_0}$. If the focal distance is large (d_i), then the magnification increases. If the focal distance is small, then the camera has a larger FoV and can image larger objects.

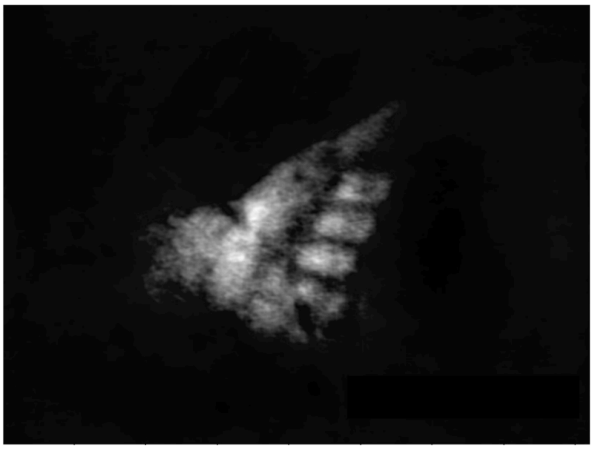


    b)
2) Caustics
    a) The caustic pattern should have: 1) thin lines and 2) lines in many directions
    b) Placing the aperture on the diffuser should make the imaging system linear shift-invariant. This means that a lateral shift of the point source causes a lateral translation of the caustic measurement. Linearity means that scaling the intensity of a point source corresponds to scaling the intensity of the sensor reading by the same amount.

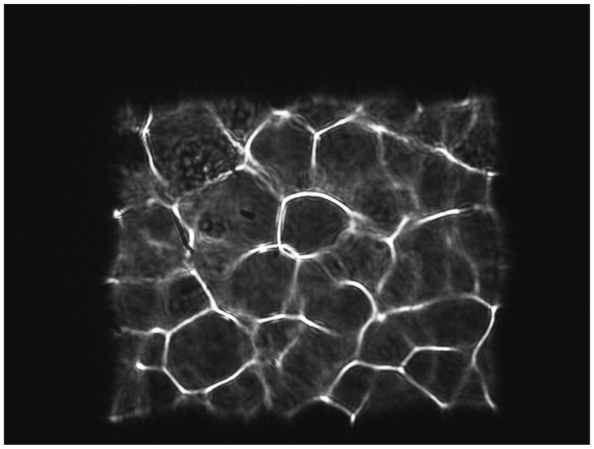
Examples



(a) PSF, with a large crop due to the small aperture



(b) Reconstructed image of a hand, 40cm away

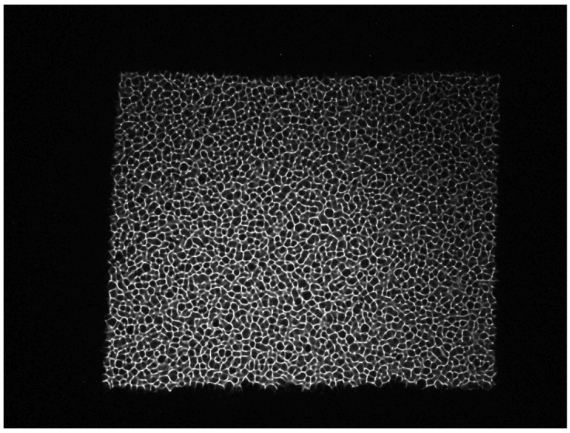


(c) PSF with appropriately sized aperture

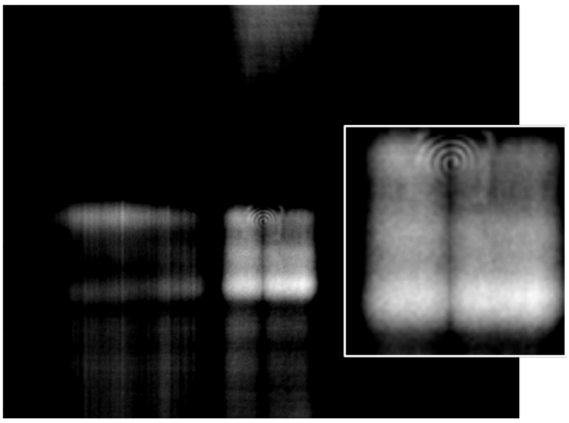


(d) Reconstructed image of a hand, 40cm away. More lines in the PSF results in a better reconstruction

- 1 degree – very small focal distance (the diffuser is basically on the sensor – generally bad for shift invariance), huge field of view, very low magnification. Need to place object super close.

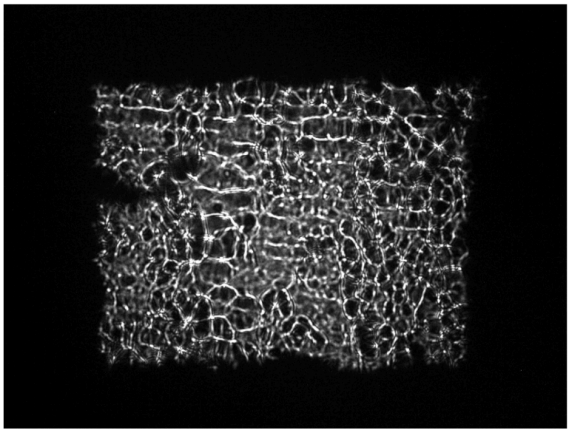


(a) PSF

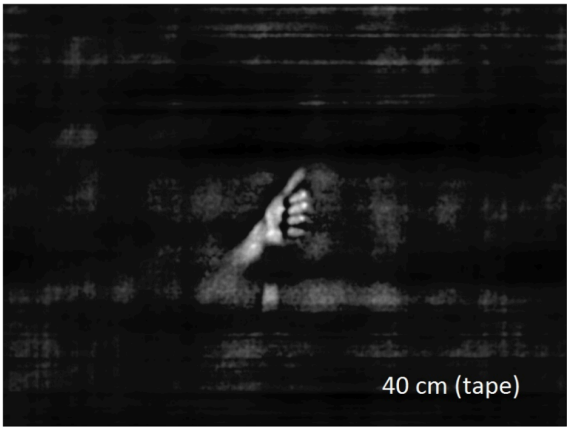


(b) Reconstructed phone image of a spiral, 7cm away (hand not possible)

- Tape – slightly larger focal distance ($\sim$ 3 mm) than 1-degree, so field of view/magnification is more manageable, but more concentrated caustics so reconstruction quality is better than the 0.5-degree.



(a) PSF



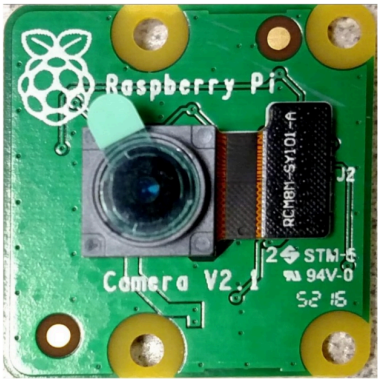
(b) Reconstructed image of a hand, 40cm away

Hardware

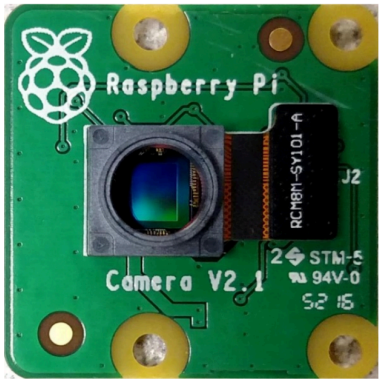
The Raspberry PiCamera is a lens screwed in a black plastic mount (Fig. 11a). We only care about the sensor, which can be accessed by unscrewing the lens (Fig. 11b). If the diffuser focal length is smaller than the height of the mount ($\sim$ 4mm) then you need to pry off the mount itself. WARNING: This is not the intended use! It's very easy to damage the camera, so be careful.

Cut the glue between the mount and the sensor with a blade (Fig. 11d). When cutting, be careful not to damage any of the electrical components on the edge of the sensor – see Fig. 11e to determine which side to start cutting from. One of the sides has fewer components.

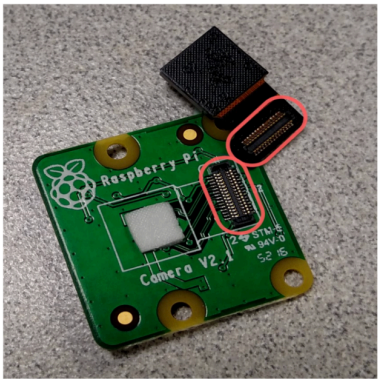
If you want, you can remove the entire sensor-mount component from the board by carefully lifting it off and breaking the glue/pad that connects it to the board. Push the sensor back onto the board once the mount is removed. Make sure the connections from the sensor circuitry to the board are fully clicked in place (Fig. 11c).



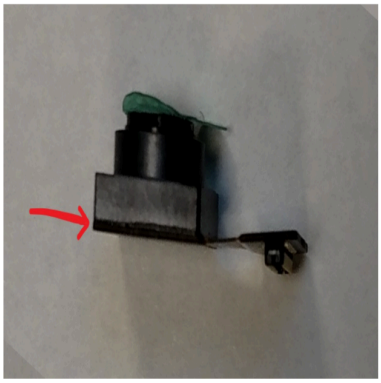
(a) PiCamera with the lens.



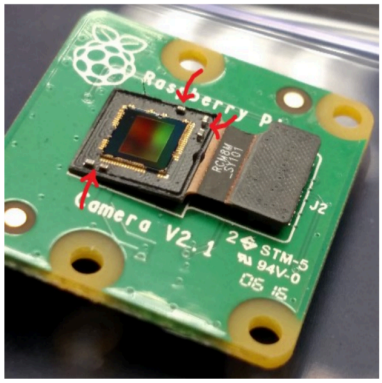
(b) Camera with the lens unscrewed.



(c) Sensor taken off the board.



(d) Sideview of the camera.



(e) Camera with the mount removed. The arrows indicate electrical compenents to avoid when cutting the glue

Figure 11: The Raspberry Pi Camera

Software

For instructions on how to download the DiffuserCam code and run the algorithms, see 4.3. These instructions are for setting up your raspberry pi and picamera in a convenient way for taking pictures and transferring the images to your computer for processing.

1. Set up the Raspberry Pi operating system (NOOBS is easiest)
2. If you don't mind always having the raspberry pi and camera connected to an external monitor and keyboard, skip to step 3. Otherwise:
   (a) Connect the Pi to wifi:
      - A general guide
      - NOTE: If you need to connect to a network that requires both a login and password (like AirBears2 on Berkeley's campus), follow the corresponding section in these instructions.

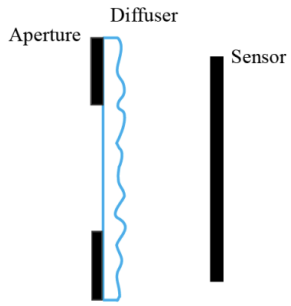
   (b) Set up VNC on your laptop and the Raspberry Pi:
      - Enable VNC Server on your Pi (or download it if your OS isn't standard).
      - Set up the VNC viewer on your laptop and connect.
      - When you connect to the pi via VNC Viewer, if the display doesn't fill the entire screen you should go to **Preferences>Raspberry Pi Configuration>Set Resolution** and set it to a resolution that allows it to fill the VNC Viewer window. Otherwise, when you use our preview code, the terminal might get obscured by the preview.

   (c) On the Raspberry Pi VNC Server, go to **Options** > **Troubleshooting** > Check the **Enable experimental direct capture mode** box. This lets the picamera preview display correctly on the VNC Viewer.
3. Install `numpy`, `pillow`, and `picamera`. The raspberry pi is very slow to assemble numpy if you use `pip`, so we recommend (note we're using `pip3` because all our code is Python3:

```
sudo apt-get install python3-picamera #Installs python3-numpy along with it
sudo apt-get install python3-pillow
```

4. Copy the `preview.py` file to your Pi.
   - If you set up VNC, then you can use the **Transfer Files** option in the VNC menu at the top of your window.
5. Connect the picamera to the appropriate slot in the Pi (it should be labeled "CAMERA") and restart your terminal.
6. Run the file via `python3 preview.py`. Follow the prompts in the terminal to change shutter speed, take the image, and name the image file.
   - With VNC, you can transfer the image back to your computer using the same file transfer option.

https://waller-lab.github.io/DiffuserCam/tutorial/algorithm_guide.pdf



Advantages over lens-based camera:

- Lightweight and smaller
- Diffuser require less precise fabrication
- Possibility of 3D imaging/microscopy and are robust to missing/dead pixels

How does it work?

- Every diffuser has a focal plane. Instead of mapping a faraway point source to a point in this plane (as lenses do), the diffuser maps a point source to a caustic pattern over the entire plane
- Notably, while the light information is spread out over the sensor, none of that information is lost.
- The point spread function (PSF) completely characterizes the scattering behavior of the diffuser *under certain assumptions*

Imaging Systems

- We can model the way a camera maps a 2D array v of light intensity values to a 2D array of pixel values on a sensor as: f(v) = b
    - We want v = f^-1(b)
    - To do so, we introduce priors to constrain the possible v's since not all maps f are injective

Forward Model f(v)

- Each ray from a point propagates to the diffuser and is locally refracted by the diffuser surface before propagating again to the sensor plane
- We must make the following approximations
    - *Shift invariance*: A lateral shift of the point source causes a lateral translation of the sensor reading
    - *Linearity*: Scaling the intensity of a point source correspond to scaling the intensity of the sensor reading by the same amount
- In other words, we assume a *linear shift-invariant* (LSI) system

Since f is linear, we can think of it like a matrix. To allow us to do this, we must vectorize the images (turn an m x n sensor reading into an mn-length vector). This trick allows us to represent the convolution as a 2D matrix M where h * v ⇔ Mv.

> v = 2D image and **v** = vectorized image

However, while all light hits the sensor plane, not all of them actually hit the physical sensor. Thus, we introduce a cropped convolution: f(v) = crop(h * v) which is represented as **CMv** = **Av** where **C** is the matrix representation of cropping and **A** = **CM**.

Inverse Problem f^-1(b)

1) The naive method is to use a Wiener deconvolution
    a) This is a common method to reverse convolution that relies on diagonalizing the measurement matrix. It cannot model the cropping behavior at all.
2) We can see that f is generally not invertible for two reasons:
    a) Information is lost in the crop operation so C is not invertible
    b) Convolving with a fixed function is not always invertible so M is not necessarily invertible
3) Instead, we can formulate this as an optimization problem which works whether A is convolutional or not:
    a) $$\mathbf{v}^* = \underset{\mathbf{v}}{\operatorname{argmin}} \frac{1}{2}\|\mathbf{A}\mathbf{v} - \mathbf{b}\|_2^2$$
        i) When **v = v***, **Av* = b**,, and the objective function **Av - b** is minimized.
4) NOTE ON SCALE: **A** is extremely large and scales with the area of the sensor. The sensor has ~10^6 pixels so **A** would have to be on the order of 10^6 * 10^6 = 10^12 entries. Therefore, any minimization algorithm must avoid loading **A** into memory.
    a) The solution is to implement the algorithm in terms of crop and convolution separately.

Solving for V

Gradient Descent

- Gradient descent is an iterative algorithm that finds the minimum of a convex function by following the slope “downhill” until it reaches a minimum.
- Instead of explicitly constructing A (the huge matrix), we compute Av and A^hv using FFTs and convolutions
    - We want v s.t. Av ~ b
        - $$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha_k \nabla g(\mathbf{x}_k),$$
    - Thus, we must optimize the function: g(v) = ½ || Av - b||^2_2
        - $$g(\mathbf{v}) = \frac{1}{2}\|\mathbf{A}\mathbf{v} - \mathbf{b}\|_2^2$$
        - $$\nabla_{\mathbf{v}} g(\mathbf{v}) = \mathbf{A}^H(\mathbf{A}\mathbf{v} - \mathbf{b}),$$

- Therefore, every GD iteration requires two operations: Av and A^H(Av - b)
  - We must find out how to efficiently compute these two operations
- Conceptually, Av = CMv = crop(h * v)
  - $$\mathbf{Av} = \mathbf{CMv}$$
    $$\mathbf{A}^H\mathbf{v} = \mathbf{M}^H\mathbf{C}^H\mathbf{v}$$
- Directly computing a convolution is expensive but the Fourier transforms give us an easy way to calculate this
  - $$\mathbf{Mv} \iff h * v = \mathcal{F}^{-1}(\mathcal{F}(h) \cdot \mathcal{F}(v)),$$
- Diag(Fh) comes from the fact that we want element-wise multiplication
  - For example:

    $$F(h) = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}, \qquad F(v) = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

    gives

    $$F(h) \cdot F(v) = \begin{bmatrix} h_1 v_1 \\ h_2 v_2 \\ h_3 v_3 \end{bmatrix}.$$

    But we can express this using normal matrix multiplication:

    $$\begin{bmatrix} h_1 & 0 & 0 \\ 0 & h_2 & 0 \\ 0 & 0 & h_3 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}.$$

    That first matrix is

    $$\operatorname{diag}(Fh).$$

    Therefore:

    $$\boxed{F(h) \cdot F(v) = \operatorname{diag}(Fh)Fv}$$
  - $$\mathcal{F}(h) \cdot \mathcal{F}(v) \iff \operatorname{diag}(\mathbf{Fh})\,\mathbf{Fv}.$$
- Now, we can bring everything together
  - $$\boxed{M = F^{-1}\operatorname{diag}(Fh)F}$$
- We can plug C back in since A = CM to get:
  - $$\boxed{A = CF^{-1}\operatorname{diag}(Fh)F}$$
- Finally, we need A^H

Recall the rule:

$$\boxed{(XYZ)^H = Z^H Y^H X^H}$$

The order **reverses**.

So:

$$A^H = F^H \left(\operatorname{diag}(Fh)\right)^H \left(F^{-1}\right)^H C^H.$$

- For a unitary Fourier transform, F^H = F^-1 so:

$$A^H = F^{-1} \left(\operatorname{diag}(Fh)\right)^H F C^H.$$

- We can simplify further by recognizing that the conjugate transpose of a diagonal matrix is the same as the complex conjugates of its entries so diag(Fh)^H = diag(Fh)*

$$\boxed{A^H = F^{-1} \operatorname{diag}(Fh)^* F C^H}$$

- What does C^H actually mean?
  - C was cropping so C^H is zero-padding
- Thus, we get these final formulas:

$$\begin{cases} \mathbf{A} = \mathbf{C}\mathbf{F}^{-1} \operatorname{diag}(\mathbf{F}\mathbf{h})\, \mathbf{F} \\ \mathbf{A}^H = \mathbf{F}^{-1} \operatorname{diag}(\mathbf{F}\mathbf{h})^*\, \mathbf{F}\mathbf{C}^H \end{cases} \iff \begin{cases} f(v) = \operatorname{crop}\left[\mathcal{F}^{-1}\left\{\mathcal{F}(h) \cdot \mathcal{F}(v)\right\}\right] \\ f^H(x) = \mathcal{F}^{-1}\left\{\mathcal{F}(h)^* \cdot \mathcal{F}(\operatorname{pad}[x])\right\}, \end{cases}$$

GD Implementation

- Zero all negative pixels in v since we can't have negative pixels in an image
- Step size a_k
  - The largest constant step size that guarantees convergence in terms of A:

$$0 < \alpha < \frac{2}{\|\mathbf{A}^H \mathbf{A}\|_2}$$ where ||A^HA||_2 is the maximum singular value of A^HA (see why)

$$\mathbf{v}_0 = I/2$$

$$\textbf{for } k = 0 \text{ to num_iters:}$$

$$\mathbf{v}'_{k+1} \leftarrow \mathbf{v}_k - \frac{1.8}{\|\mathbf{A}^H \mathbf{A}\|} \mathbf{A}^H (\mathbf{A}\mathbf{v}_k - \mathbf{b})$$

$$\mathbf{v}_{k+1} \leftarrow \underset{\mathbf{v} \geq 0}{\operatorname{proj}}(\mathbf{v}'_{k+1})$$

Gradient Descent Speedups

Accelerated projected gradient descent

- A momentum term determines how much of the new descent direction is determined by the old descent direction. We want to guard against changing the descent direction too much and too often.
    - u is the momentum term
    - P_0 is Nesterov momentum which involves an intermediate update p

$$\mathbf{v}_0 = I/2, \quad \mu = 0.9, \quad \mathbf{p}_0 = 0$$
$$\textbf{for } k = 0 \text{ to num_iters:}$$
$$\mathbf{p}_{k+1} \leftarrow \mu \mathbf{p}_k - \alpha_k \mathbf{grad}(\mathbf{v}_k)$$
$$\mathbf{v}'_{k+1} \leftarrow \mathbf{v}_k - \mu \mathbf{p}_k + (1+\mu)\mathbf{p}_{k+1}$$
$$\mathbf{v}_{k+1} \leftarrow \underset{\mathbf{v} \geq 0}{\text{proj}}(\mathbf{v}'_{k+1})$$

    -

FISTA (fast iterative shrinkage-thresholding algorithm)

- Computes accelerated projected gradient descent but is more flexible about what the projection (proximal) step does

$$\mathbf{v}_0 = I/2, \quad t_1 = 1, \quad x_0 = \mathbf{v}_0$$
$$\textbf{for } k = 0 \text{ to num_iters:}$$
$$x_k \leftarrow p_L(\mathbf{v}_k)$$
$$t_{k+1} \leftarrow \frac{1 + \sqrt{1 + 4t_k^2}}{2}$$
$$\mathbf{v}_{k+1} \leftarrow x_k + \frac{t_k - 1}{t_{k+1}}(x_k - x_{k-1})$$

-
- (see this paper)

ADMM

- *Gradient descent tries to solve the whole reconstruction problem at once. ADMM breaks it into several smaller, easy problems, solves those separately, and then forces their answers to agree.*
- Naively, we have three three variables: C crop, M convolution, and v >=0 all tied to variable v
- Let's define three helper variables:

    - x = Mv
    - u = phi v
    - w = v
- Thus, the measurement model becomes b ~ Cx instead of b ~ CMv
- Phi is the derivative operator
- W is a copy used for non-negativity and has w >= 0 imposed onto it
    - Instead of forcing v >= 0 inside every calculation, ADMM gives that constraint its own variable
- Then, the minimization problem becomes
    - $$\min_{w\geq 0,u,x,v} \frac{1}{2}\|b - Cx\|_2^2 + \tau\|u\|_1$$

      subject to

      $$x = Mv, \qquad u = \Psi v, \qquad w = v.$$
    - $$\begin{aligned} x &: \text{match the measured sensor data} \\ u &: \text{make the image gradient sparse} \\ w &: \text{make the image nonnegative} \\ v &: \text{actual reconstructed image} \end{aligned}$$
- However, Mv and x can differ now. Thus, we add an *augmented Lagrangian* to penalize the optimization for making x disagree with Mv

Suppose all we wanted was

$$x = Mv.$$

ADMM adds a penalty

$$\boxed{\frac{\mu_1}{2}\|Mv - x\|^2}$$

to the objective.

If

$$Mv = x,$$

the penalty is zero.

If they're far apart,

$$\|Mv - x\|^2$$

gets large.

So minimizing the objective naturally pushes

$$Mv \to x.$$

- ADMM gives each equality constraint a dual variable which are running records of how badly each constraint has been violated
- $$\begin{array}{rcl} x & : & \text{match the measured sensor data} \\ u & : & \text{make the image gradient sparse} \\ w & : & \text{make the image nonnegative} \\ v & : & \text{actual reconstructed image} \end{array}$$

$$\begin{aligned} Mv = x &\quad\longleftrightarrow\quad \xi \\ \Psi v = u &\quad\longleftrightarrow\quad \eta \\ v = w &\quad\longleftrightarrow\quad \rho. \end{aligned}$$

- Functionally, these dual variables act as a way for the optimizer to maintain memory of constraint violation

$$
\begin{aligned}
L = & \frac{1}{2}\|b - Cx\|^2 + \tau\|u\|_1 \\
& + \frac{\mu_1}{2}\|Mv - x\|^2 + \xi^H(Mv - x) \\
& + \frac{\mu_2}{2}\|\Psi v - u\|^2 + \eta^H(\Psi v - u) \\
& + \frac{\mu_3}{2}\|v - w\|^2 + \rho^H(v - w) \\
& + 1_+(w).
\end{aligned}
$$

- Basically, v has three constraints:
  - When convolved with the PSF, it should look like x
  - Its gradients need to look like sparse gradients u
  - It must agree with the non-negative image w

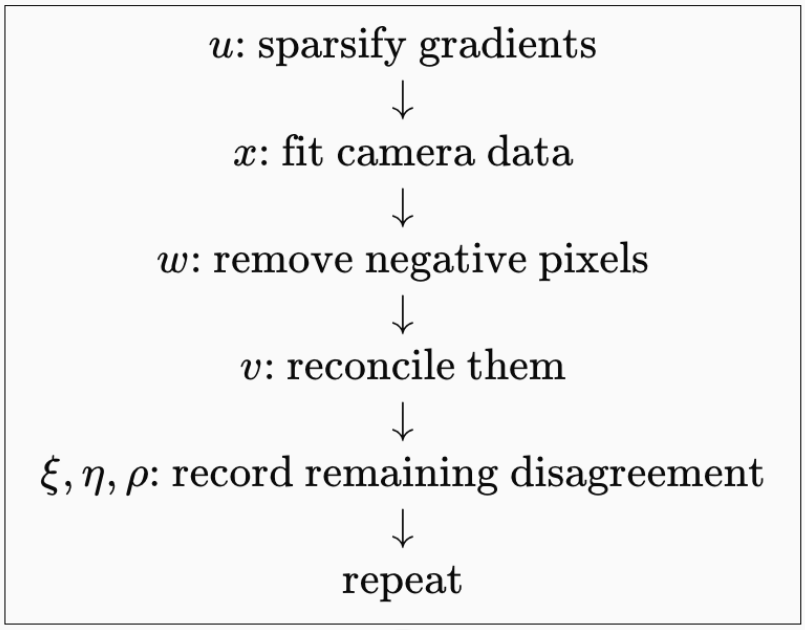
Then the dual variables are referees:

$$\xi : \quad \text{"}Mv \text{ and } x \text{ still disagree."}$$

$$\eta : \quad \text{"}\Psi v \text{ and } u \text{ still disagree."}$$

$$\rho : \quad \text{"}v \text{ and } w \text{ still disagree."}$$

Every iteration:



-
-

[2111.06377] Masked Autoencoders Are Scalable Vision Learners

MAE-Style Masked-Patch Inpainting VS Convolutional Inpainting
Restore heavily occluded image patches and compare against convolutional impainting baseline

Research Question: Does the transformer's global self-attention outperform the CNN's local bias? And how does the performance gap change as the mask ratio changes?

Hypothesis: